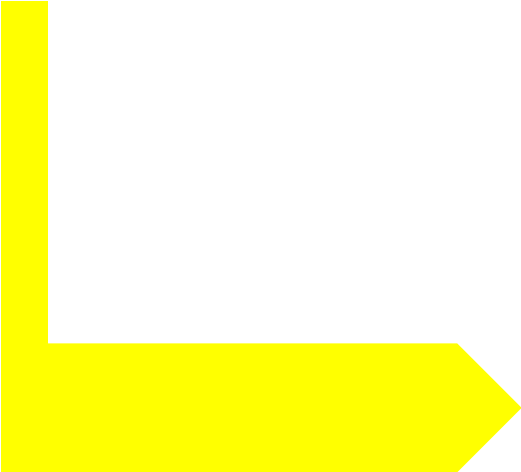


A thick yellow vertical bar on the left side of the page, with a horizontal yellow arrow pointing to the right, intersecting the vertical bar.

Emotion Recognition

A series of thin, curved lines in shades of yellow and blue, originating from the bottom left and extending upwards and to the right, resembling stylized grass or reeds.

Conte Gabriele
D'Agostino Pierantonio
De Giosa Rosangela
Leo Daniele
Troccoli Tommaso Gabriele



**Progetto d'esame per il corso di Ingegneria del
Software
– Anno Accademico 2024/2025-**

Progetto di Apprendistato di Alta Formazione, in
collaborazione tra **EY** e il **Politecnico di Bari**.

Il lavoro è stato sviluppato dagli studenti
partecipanti al programma, con il supporto del tutor
aziendale
Pasquale Donato Roberto.

Contents

Il Team	3
Introduzione	4
Obbiettivi	4
Utenti target dell'applicazione	4
Timeline di Sviluppo:	5
Stato dell'Arte	6
Analisi di Mercato	6
Project Management	7
Analisi dei rischi	7
Analisi dei Costi	9
FURPS+	11
Diagramma delle Classi	12
Il progetto	15
Metodo proposto e requisiti	15
Technology Stack	15
Diagramma delle Componenti	16
Struttura di Deployment	18
Flusso delle Operazioni	20
Livello Front-end: Interfaccia Utente	21
Livello Back-end: Logica Applicativa	22
Struttura Server e Deployment	22
Vantaggi del Tech Stack Selezionato	23
Testing	24
Validazione dell' Applicativo Run-Time	26
Home Page	26
Aspetti tecnici e di design	29
Profile Page	30
Image Page	33
FINAL PAGE	37
Conclusioni e sviluppi futuri	39
Bibliografica	40

Il Team

Il team di sviluppo dell'applicativo è composto da:



Conte Gabriele
Gabriele.Conte@it.ey.com



D'Agostino Pierantonio
Pierantonio.DAgostino@it.ey.com



De Giosa Rosangela
Rosangela.De.Giosa@it.ey.com



Leo Daniele
Daniele.Leo@it.ey.com



Troccoli Tommaso Gabriele
Tommaso.Gabriele.Troccoli@it.ey.com

Introduzione

Emotion Recognition è una web application innovativa, orientata alla ricerca, progettata per analizzare le emozioni umane in risposta a stimoli visivi, attraverso l'integrazione di tecnologie di face analysis, Natural Language Processing (NLP) e una User Experience guidata.

L'app consente agli utenti di creare un profilo personale ed essere guidati in un percorso interattivo. Durante l'esperienza, vengono mostrate una serie di immagini pensate per evocare specifiche emozioni. Per ciascuna immagine, l'utente è invitato a:

- selezionare l'emozione percepita tra quelle disponibili,
- fornire una breve descrizione testuale del proprio stato emotivo,
- consentire la registrazione video tramite webcam per il rilevamento automatico delle espressioni facciali.

L'applicazione utilizza la fotocamera frontale per acquisire in tempo reale le reazioni emotive dell'utente, elaborandole tramite tecniche avanzate di riconoscimento facciale. I dati raccolti – emozioni dichiarate, descrizioni testuali e segnali visivi, vengono successivamente elaborati da un modello di Intelligenza Artificiale, al fine di generare un'analisi emotiva approfondita e personalizzata.

Obiettivi

L'applicazione è stata progettata con l'obiettivo di supportare attività di ricerca nel campo dello studio delle emozioni, rendendosi adatta all'impiego in contesti accademici, clinici e sperimentali.. Consente la raccolta di dati sia demografici sia emozionali, costruendo così un bacino informativo utile per svolgere analisi statistiche e comportamentali. Inoltre, l'app mette a disposizione strumenti standardizzati per la valutazione delle emozioni, al fine di garantire coerenza e affidabilità nei risultati ottenuti. Una funzionalità chiave dell'applicazione è la possibilità di registrare video delle reazioni emotive degli utenti in tempo reale, offrendo così un ulteriore livello di approfondimento qualitativo nell'analisi dei dati.

Utenti target dell'applicazione

L'applicazione è rivolta a un pubblico specifico, composto da utenti che operano, studiano o sono coinvolti in ambiti legati alla psicologia, alle neuroscienze e all'analisi dei dati.

I principali destinatari sono i partecipanti a studi psicologici, che attraverso l'utilizzo della piattaforma contribuiscono attivamente alla raccolta di dati emozionali e comportamentali.

Un secondo gruppo target è costituito da ricercatori e studenti impegnati in attività accademiche nei settori della psicologia e delle neuroscienze, per i quali l'app rappresenta uno strumento utile sia per la raccolta sia per l'analisi dei dati.

Anche i data scientist che lavorano su progetti di riconoscimento delle emozioni possono beneficiare delle funzionalità dell'app, soprattutto in termini di disponibilità di dati strutturati e integrazione con strumenti di elaborazione automatica.

Il servizio, infine, è anche pensata per tutti gli utenti non specialisti comunque motivati a contribuire attivamente alla ricerca scientifica nel campo delle emozioni.

Timeline di Sviluppo:

- **31 Marzo- 4 Aprile 2025:** Analisi, pianificazione e definizione degli obiettivi progettuali.
- **7 Aprile- 11 Aprile 2025:** Setup iniziale e sviluppo base.
- **14 Aprile – 18 Aprile 2025:** Sviluppo flusso utente ed implementazione sistema di autenticazione.
- **21 Aprile- 25 Aprile 2025:** Integrazione con WebRTC.
- **28 Aprile- 7 Maggio 2025:** Integrazione con API esterne per il riconoscimento delle emozioni e gestione ed invio dei risultati al backend.
- **8 Maggio- 12 Maggio 2025:** Gestione degli errori lato frontend e backend.
- **13 Maggio- 19 Maggio 2025:** Containerizzazione dell'intera applicazione tramite Docker e configurazione ambiente di produzione.
- **20 Maggio – 27 Maggio 2025:** Completamento attività di ottimizzazione e performance di test responsive.
- **19 Maggio- 1 Giugno 2025:** Rifinitura documentazione tecnica e funzionale.

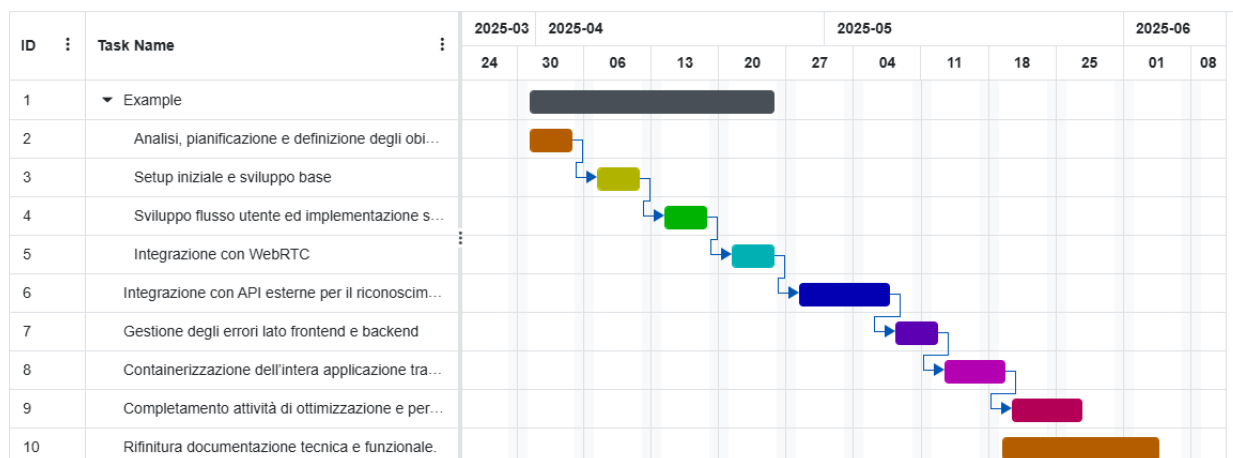


Figura 1: Diagramma di Gantt

Stato dell'Arte

Analisi di Mercato

Emotion Recognition si inserisce in un panorama tecnologico in forte crescita, collocandosi all'intersezione tra analisi emotiva, Intelligenza Artificiale e strumenti digitali per la ricerca scientifica.

Il crescente interesse verso le tecnologie in grado di rilevare e interpretare le emozioni umane, noto anche come *affective computing*, ha reso questo settore un campo di applicazione strategico sia in ambito accademico sia in contesti clinici e sperimentali.

Negli ultimi anni, il mercato globale delle tecnologie per il riconoscimento delle emozioni ha registrato una significativa espansione, trainata da fattori quali l'evoluzione delle tecniche di computer vision, l'integrazione di modelli NLP nella comprensione del linguaggio emotivo e il crescente utilizzo di sistemi AI nei settori della salute mentale, dell'istruzione e della ricerca.

Secondo report internazionali, si stima che il mercato del riconoscimento emozionale supererà i 90 miliardi di dollari entro il 2030, con tassi di crescita annui superiori al 15%, trainati principalmente dalla domanda nei settori della customer experience, della psicologia digitale e dell'health tech.

In questo scenario, Emotion Recognition si propone come una piattaforma innovativa, orientata non al mercato consumer bensì alla raccolta e analisi dei dati in contesti controllati.

L'applicazione si distingue per l'integrazione simultanea di tre fonti informative: le emozioni dichiarate dall'utente, le descrizioni testuali e le espressioni facciali rilevate in tempo reale tramite webcam. Questo approccio multimodale costituisce un elemento differenziante rispetto a molte soluzioni esistenti, che spesso si limitano a una singola modalità (audio, video o testo).

Il segmento di riferimento di Emotion Recognition è ben definito: ricercatori, studenti universitari, psicologi sperimentali e data scientist rappresentano il nucleo degli utenti target, accomunati dall'interesse per strumenti digitali avanzati di indagine emotiva. L'applicazione si configura, quindi, come un tool utile per progetti di ricerca, tesi sperimentali, studi clinici e attività didattiche incentrate sull'analisi del comportamento umano.

Dal punto di vista competitivo, Emotion Recognition si differenzia da piattaforme commerciali di *emotion tracking* orientate all'advertising o alla user experience aziendale. Il suo posizionamento è quello di una web application a vocazione scientifica, caratterizzata da un'interfaccia guidata, strumenti di analisi standardizzati e un'architettura che facilita l'acquisizione di dati strutturati e anonimizzati, compatibili con protocolli di ricerca e norme etiche.

Inoltre, l'adozione del cloud (tramite Azure Blob Storage) e l'uso di tecnologie avanzate per il riconoscimento facciale e il NLP, rendono la piattaforma scalabile e potenzialmente estendibile anche ad altri contesti applicativi, come la valutazione dello stress, la formazione in ambito clinico o l'integrazione in ambienti virtuali e simulazioni psicologiche.

In sintesi dunque, Emotion Recognition si colloca in un mercato in espansione, caratterizzato da una forte richiesta di strumenti di analisi emozionale intelligenti, e rappresenta un prodotto che, pur nascendo in ambito universitario, mostra potenzialità concrete di applicazione nel mondo della ricerca scientifica e della sperimentazione clinica. La natura modulare dell'architettura e il focus sulla qualità del dato raccolto

pongono le basi per un'evoluzione futura dell'applicativo verso scenari di utilizzo più ampi, sempre all'interno di un framework etico e metodologico rigoroso.

Project Management

Analisi dei rischi

Nel corso dello sviluppo e della distribuzione dell'applicazione sono stati individuati diversi potenziali rischi, articolati su più livelli: tecnico, legale, etico e gestionale. Una valutazione preventiva di tali criticità è essenziale per garantire l'affidabilità, la sicurezza e la conformità dell'intero sistema.

1. Rischi legati alla gestione dei dati sensibili

L'applicazione raccoglie e gestisce dati emotivi, demografici e, potenzialmente, biometrici (come espressioni facciali rilevate da webcam). Questi rientrano nella categoria dei dati personali sensibili secondo il GDPR e normative simili. Un'inadeguata protezione, come l'assenza di cifratura lato server, una gestione errata dei token di accesso o vulnerabilità nell'autenticazione, potrebbe generare gravi responsabilità legali ed etiche. La conformità deve essere garantita attraverso policy di data retention, sistemi di anonimizzazione e consenso informato esplicito.

2. Rischi tecnici e di affidabilità del sistema

Durante le fasi di acquisizione ed elaborazione, sono emerse criticità di tipo tecnico che possono compromettere l'esperienza utente e l'integrità dei dati raccolti. I problemi ricorrenti includono:

- Webcam malfunzionanti o assenti: è stato necessario implementare controlli via `navigator.mediaDevices.enumerateDevices()` e fallback per dispositivi non supportati. Il metodo `getUserMedia()` è stato spesso oggetto di errori di permesso, incompatibilità o assenza di input, soprattutto in ambienti mobili o browser obsoleti.
- Problemi nell'integrazione delle API: in fase di deploy, l'app ha mostrato instabilità nei collegamenti con API esterne per la classificazione emozionale. Sono state adottate strategie di fallback come test periodici degli endpoint di "health check", analisi dettagliata dei codici HTTP e log degli errori per identificare problemi di latenza, throttling o autenticazione.
- Gestione dello stato condiviso: nei moduli React, errori nella propagazione del contesto globale o race condition tra gli aggiornamenti asincroni dello stato hanno causato malfunzionamenti in alcune sessioni utente. È stato necessario ricorrere a `useEffect` per monitorare transizioni critiche e validare l'integrità dello stato a ogni aggiornamento.
- Instabilità delle connessioni WebSocket: la comunicazione in tempo reale è stata soggetta a problematiche comuni quali:
 - *Connection Refused*: in ambienti locali o in produzione a causa di errori nella porta di esposizione del server WebSocket.

- *CORS o problemi TLS*: incompatibilità tra ws:// e wss:// in base al protocollo HTTPS della pagina frontend.
- *Firewall o proxy aziendali*: inaccessibilità della connessione per restrizioni di rete.
- *Limiti del browser*: alcuni browser impongono restrizioni sul numero di connessioni concorrenti.

Le azioni correttive hanno incluso: test manuali via WebSocket client, logging avanzato degli eventi, configurazione delle variabili d'ambiente per URL dinamiche e implementazione di meccanismi di keep-alive.

3. Rischi legati all'accuratezza dell'analisi emozionale

I modelli di intelligenza artificiale impiegati per l'interpretazione delle emozioni, sia a partire da immagini sia da testo, presentano limiti intrinseci. In particolare:

- l'accuratezza può degradare significativamente in condizioni di scarsa illuminazione, inquadrature parziali o espressioni ambigue;
- i modelli NLP per l'analisi del sentiment possono non cogliere sarcasmo, ironia o ambivalenze linguistiche;
- persistono bias algoritmici legati a dati di training sbilanciati per etnia, genere o fascia d'età.

Tali problematiche possono compromettere la validità scientifica delle analisi se non adeguatamente mitigati tramite validazione sperimentale e/o feedback umano.

4. Rischi legati all'esperienza utente

L'usabilità dell'interfaccia gioca un ruolo centrale nella qualità dei dati raccolti. Interfacce non intuitive, processi di onboarding poco chiari o tempi di attesa elevati (soprattutto durante l'elaborazione AI) possono portare a interazioni incomplete o abbandoni. Per mitigare questi rischi sono stati utilizzati strumenti come:

- React DevTools per ispezione dei componenti e profiling;
- Browser DevTools per analisi delle richieste di rete e degli errori in tempo reale;
- Web Vitals per il monitoraggio dei tempi di caricamento e reattività;
- Analisi del bundle per ottimizzare il caricamento e la suddivisione del codice in chunk.

5. Rischi organizzativi e di gestione del progetto

Come in ogni progetto, vi sono rischi di natura gestionale: ritardi dovuti a dipendenze tecnologiche esterne, modifica dei requisiti funzionali in corso d'opera, o scarsa sincronizzazione tra i membri del team (sviluppatori, ricercatori, esperti UX). Il coordinamento è stato affrontato tramite strumenti di project management (es. GitHub Projects) e metodologie agili a sprint brevi, ma permane una certa fragilità in ambienti a risorse limitate.

IMPATTO/ PROBABILITA'	Rare	Unlikely	Possible	Likely	Almost certain
Catastrophic	moderate	moderate	Rischi legati alla gestione dei dati sensibili	critical	critical
Major	low	moderate	moderate	Rischi tecnici e di affidabilità del sistema	critical
Moderate	low	moderate	Rischi legati all'esperienza utente	Rischi legati all'accuratezza dell'analisi emozionale	high
Minor	very low	low	Rischi organizzativi e di gestione del progetto	moderate	moderate
Insignificant	very low	very low	low	low	moderate

Figura 2: Matrice di Rischio

Analisi dei Costi

Il presente progetto è stato sviluppato utilizzando la sottoscrizione Azure for Students, un programma educativo di Microsoft che fornisce agli studenti accesso gratuito ai servizi cloud di Azure.

Il piano prevede un credito iniziale di \$100 da utilizzare nei primi 12 mesi, oltre all'accesso a più di 25 servizi gratuiti, tra cui compute, networking, storage e database. Tutti i costi sostenuti durante questo periodo vengono detratti dal credito disponibile, consentendo agli studenti di sperimentare e sviluppare progetti reali senza costi aggiuntivi.

Il progetto presenta un costo totale accumulato di €23,63 al 4 giugno 2025, con una previsione di spesa che raggiungerà €121,73 entro la fine del mese. Il trend di crescita mostrato nel grafico temporale indica un'accelerazione significativa dei consumi nelle settimane successive, suggerendo un incremento dell'utilizzo delle risorse o l'attivazione di servizi aggiuntivi.

L'analisi della ripartizione per servizio evidenzia le seguenti componenti principali:

- SQL Database (€18,93 - 80% del totale): Rappresenta la voce di costo predominante, indicando l'utilizzo di Azure SQL Database per la gestione dei dati relazionali del progetto. Questo servizio comprende i costi di compute, storage e licenze associate all'istanza database, configurata presumibilmente per supportare operazioni di analisi dei dati complesse.
- App Service (€4,29 - 18% del totale): Corrisponde ai costi di hosting delle applicazioni web tramite Azure App Service, includendo le risorse di calcolo dedicate all'esecuzione delle applicazioni e dei servizi web. Questo servizio fornisce una piattaforma completamente gestita per il deployment di applicazioni native e web.
- Virtual Network (€0,36 - 1,5% del totale): Rappresenta i costi associati alla configurazione e gestione della rete virtuale, inclusi eventuali gateway, load balancer o servizi di connettività avanzati necessari per l'interconnessione sicura dei servizi.

Servizi a costo zero:

- Azure DNS (€0,05): Servizio di risoluzione nomi di dominio

- Storage (€0,00): Account di archiviazione per blob, file e dati
- Bandwidth (€0,00): Trasferimento dati in uscita
- Container Registry (€0,00): Registro per immagini container Docker

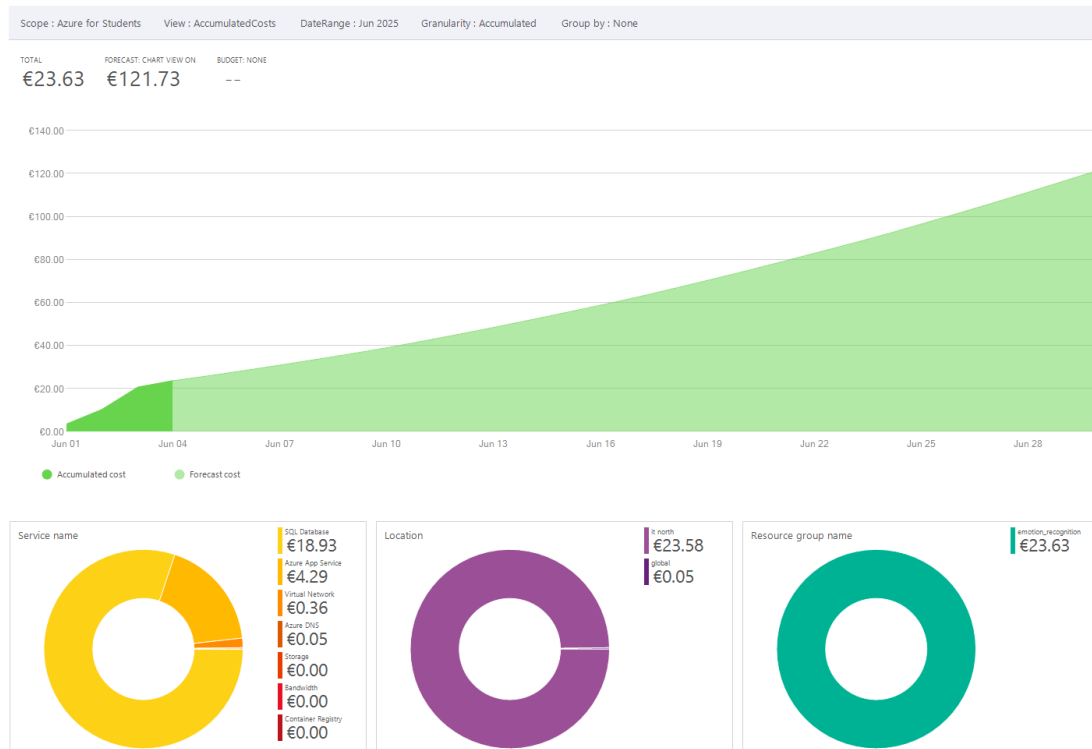


Figura 3: Grafico relativo all'andamento dei costi progettuali

FURPS+

Per valutare la qualità complessiva dell'applicazione, è stato adottato il modello **FURPS+**, che considera diversi aspetti fondamentali per garantire un prodotto robusto e usabile.

- **Functionality (Funzionalità):** L'app fornisce un flusso completo per la registrazione delle emozioni, inclusa la raccolta di dati demografici, l'integrazione con webcam per la registrazione video e l'analisi tramite API di intelligenza artificiale. Le funzionalità sono state sviluppate per assicurare la facilità d'uso e la copertura delle esigenze dei ricercatori.
- **Usability (Usabilità):** L'interfaccia utente è progettata con un approccio mobile-first, garantendo accessibilità e facilità d'interazione sia da desktop che da dispositivi mobili, grazie all'uso di Bootstrap 5 e componenti React.
- **Reliability (Affidabilità):** Il sistema include meccanismi di gestione degli errori, autenticazione sicura via JWT, e strategie di retry automatico per garantire stabilità durante l'uso.
- **Performance (Prestazioni):** L'applicazione è ottimizzata per caricare rapidamente le pagine e gestire in modo efficiente la registrazione video e l'upload, minimizzando l'uso delle risorse di sistema.
- **Supportability (Manutenibilità):** L'architettura modulare, l'uso di React Context per la gestione dello stato globale e la containerizzazione tramite Docker favoriscono la manutenzione e la scalabilità futura del progetto.

Diagramma delle Classi

Il modello dei dati dell'applicazione è stato progettato con un'architettura multilivello orientata a garantire coerenza, scalabilità e integrità nella gestione strutturata delle informazioni relative agli utenti, alle loro risposte emozionali e ai processi di elaborazione automatizzata supportati da modelli di Intelligenza Artificiale.

La persistenza dei dati è affidata a un database relazionale PostgreSQL, il quale assicura vincoli di integrità referenziale, come ad esempio quelli tra utenti e reazioni, e risulta pienamente compatibile con ambienti cloud ad alta disponibilità.

Le entità fondamentali del dominio sono modellate come classi Entity corrispondenti alle tabelle del database, mentre le componenti intermedie, come DTO, servizi e controller, svolgono un ruolo di mediazione tra i livelli di persistenza e il frontend applicativo.

La gestione degli utenti è affidata alla classe **UserEntity**, che rappresenta l'anagrafica dell'utente nel database. Tale entità è caratterizzata da un identificativo numerico primario e da campi relativi all'identità digitale (quali email e password hashata) e a informazioni personali come nickname, età, genere e nazionalità. Le operazioni di accesso ai dati su questa entità sono incapsulate nel **UserRepository**, responsabile delle operazioni CRUD.

Il servizio **UserService** si occupa della logica applicativa associata alla gestione dell'autenticazione e dell'interazione utente. Le sue principali responsabilità includono il login, attraverso cui autentica l'utente e genera JWT, e il logout, che revoca i token attivi mediante la loro registrazione in blacklist. Inoltre, il servizio è incaricato della creazione dei token JWT firmati tramite il metodo `packJwts`.

Questa logica di autenticazione è fortemente integrata in componenti come **JwtProvider**, per la gestione dei token, **BlacklistTokenRepository** per l'invalidazione dei token durante il logout, e **AzureStorageService**, utilizzato per l'accesso alle immagini archiviate su Azure Blob Storage.

Il servizio **AzureStorageService** consente il caricamento dinamico delle immagini da sottoporre agli utenti. Tra le sue funzionalità principali vi è la selezione casuale di immagini, il recupero di specifici blob come array di byte e la possibilità, lato amministrativo, di elencare tutte le risorse disponibili. Tutti i contenuti sono memorizzati all'interno di contenitori protetti di Azure Blob e accessibili solo tramite richieste autorizzate.

La classe **ReactionsEntity** rappresenta la risposta emozionale fornita da un utente a una determinata immagine. Ogni istanza di questa entità corrisponde a una reazione singola, associata a una sessione specifica.

Essa è relazionata a **UserEntity** tramite una relazione molti-a-uno, riflettendo il fatto che un utente può registrare più reazioni nel tempo. I principali attributi comprendono l'identificativo della reazione, il riferimento all'utente, l'identificativo o path dell'immagine somministrata, l'emozione selezionata, una descrizione testuale dell'esperienza emotiva e una valutazione generata dal sistema AI.

Nel flusso dati del sistema, le reazioni vengono trasmesse al backend all'interno di un oggetto aggregato chiamato **ResultPayload**. Questo oggetto contiene una lista di **ImageDescriptionAndReactionPayload**, ognuno dei quali incapsula la risposta a una singola immagine.

La classe **ReactionsService**, responsabile della logica di business legata alla registrazione delle reazioni emozionali, si occupa della validazione del riferimento all'utente, della trasformazione dei payload ricevuti in entità persistenti e del loro salvataggio in batch tramite il **ReactionsRepository**.

Il corretto funzionamento del servizio viene validato attraverso test unitari nella classe **ReactionsServiceTest**, che copre casi di successo e di errore, come l'assenza dell'utente.

L'emozione selezionata dall'utente viene trattata come una enumerazione, **ReactionEnum**, che garantisce la consistenza dei valori, facilita la serializzazione e la deserializzazione tra frontend e backend e consente una validazione automatica dei dati in ingresso mediante `valueOf`.

Al momento del login, il sistema restituisce un oggetto **LoginResponse**, che include l'identificativo dell'utente autenticato e la lista delle immagini da presentare, selezionate dinamicamente dal servizio `AzureStorageService`.

Per quanto riguarda la sicurezza, l'autenticazione si basa su JWT (JSON Web Token), gestiti in modo modulare. Il componente **JwtProvider** rappresenta la classe centrale per la creazione, validazione e decodifica dei token, esponendo metodi per l'estrazione di informazioni rilevanti come username, data di scadenza e claims, oltre che per la validazione e la verifica di integrità. Il **JwtAuthFilter** intercetta tutte le richieste HTTP in ingresso e applica controlli di autenticazione, verificando la presenza e validità del token e restituendo eventuali errori tramite la costruzione di una risposta 401.

Il comportamento globale della sicurezza è definito nella classe **SecurityConfigurer**, la quale configura la catena dei filtri di Spring Security, distinguendo tra rotte pubbliche e protette, e gestendo aspetti come le policy CORS, CSRF e l'integrazione del **JwtAuthFilter**.

Tutti i componenti della sicurezza sono testati attraverso le classi **JwtProviderTest**, **JwtAuthFilterTest** e **SecurityConfigurerTest**, che coprono scenari di token scaduti, mancanti o non validi, nonché l'accesso a risorse pubbliche e protette.

Nel processo di raccolta e restituzione dei dati emozionali, il sistema fa uso di DTO progettati per il trasferimento via REST API. Il DTO **ImageDescriptionAndReactionPayload** incapsula i dati relativi alla reazione a una singola immagine, inclusi identificatore, descrizione testuale, emozione selezionata e valutazione AI. Il contenitore aggregato **ResultPayload** raccoglie una lista di questi oggetti e viene restituito alla fine della sessione per consentire all'utente la visualizzazione di un riepilogo personalizzato. La sicurezza dei token JWT viene ulteriormente rafforzata da un meccanismo di blacklist, che consente la loro invalidazione prima della naturale scadenza. Questo è particolarmente utile in casi di logout, cambio di credenziali o sospetti di uso illecito. Il modello dati prevede un'entità **BlacklistTokenEntity**, che rappresenta il token revocato con relativo identificatore e data di scadenza. L'accesso ai token revocati è gestito dal **BlacklistTokenRepository**, mentre la verifica in tempo reale avviene attraverso il **BlacklistJwtFilter**, che esamina ogni richiesta in ingresso per verificare se il JWT è presente e non è stato invalidato. In caso contrario, il filtro blocca la richiesta, altrimenti ne consente il passaggio alla catena successiva. Il comportamento del filtro è validato nella classe **BlacklistJwtFilterTest**, che contempla numerosi scenari, come la validità del token, l'assenza di token o l'accesso a risorse pubbliche.

Il controller REST **AppController** espone infine gli endpoint principali dell'applicazione, come quelli per login, logout, registrazione delle reazioni, download delle immagini e recupero dati di benvenuto. Il comportamento del controller e dei servizi associati è testato attraverso **AppControllerTest** e **UserServiceTest**, che simulano vari scenari operativi, inclusi accessi autorizzati, tentativi con token scaduti o assenti, problemi nel caricamento delle immagini e tentativi multipli di login.

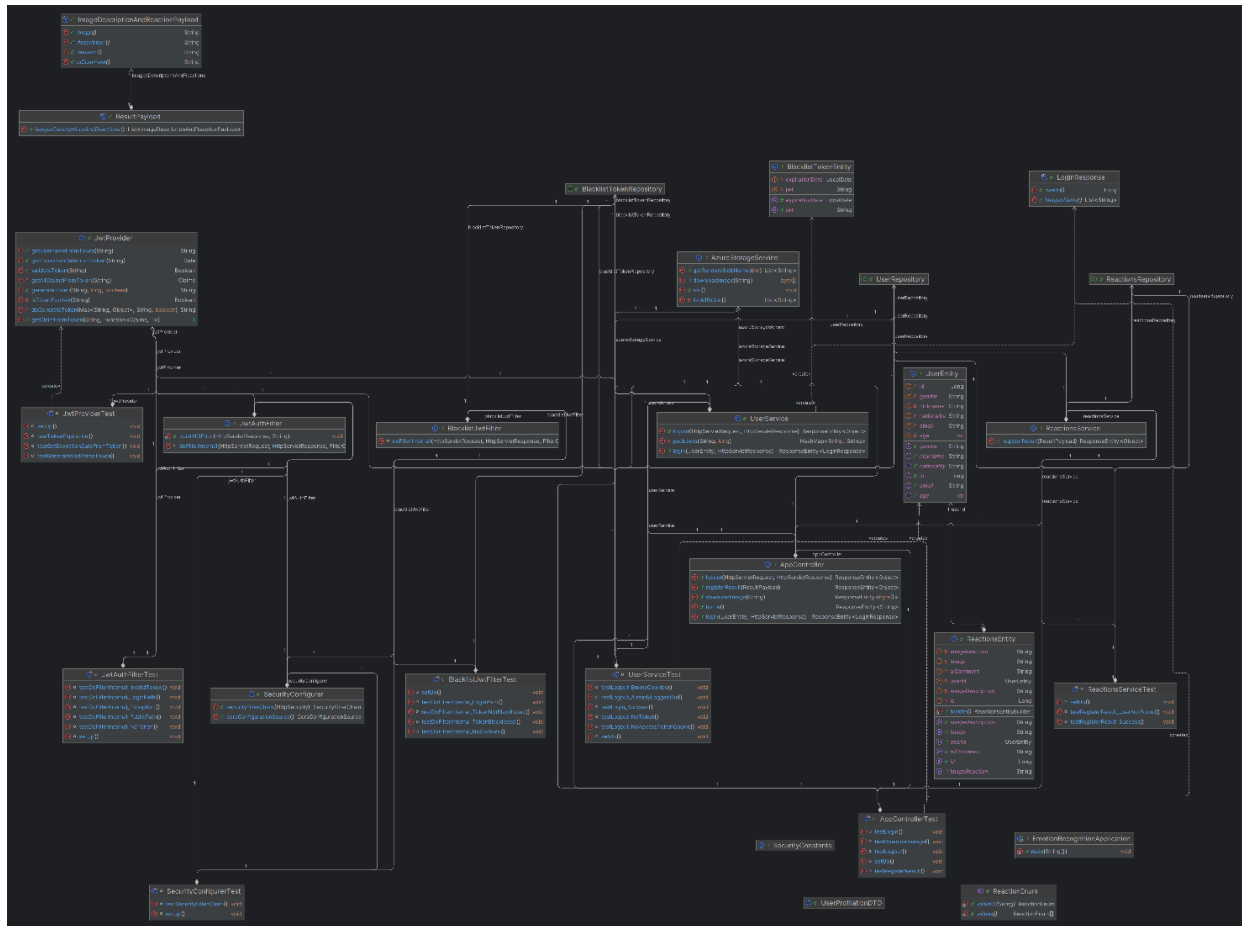


Figura 4: Diagramma delle classi

Il progetto

Metodo proposto e requisiti

Il progetto **Emotion Recognition** è stato sviluppato per supportare studi psicologici e neuroscientifici nel riconoscimento e nell'analisi delle emozioni tramite l'utilizzo di tecnologie avanzate di intelligenza artificiale e video registrazione. Il metodo proposto prevede un flusso di lavoro multi-step, che accompagna l'utente attraverso diverse fasi: dalla creazione del profilo personale, alla visione di immagini specifiche, fino alla registrazione e analisi delle reazioni emotive in tempo reale. Questo processo è supportato da interfacce intuitive e da un sistema di backend sicuro che garantisce la raccolta e l'elaborazione dei dati necessari per la ricerca. I requisiti funzionali si concentrano sull'accuratezza nella cattura delle reazioni facciali, la sicurezza dei dati raccolti e l'affidabilità delle analisi emotive. Requisiti non funzionali, come la responsività dell'interfaccia, la compatibilità cross-browser e la scalabilità del sistema, sono stati considerati fin dalle prime fasi progettuali per assicurare una fruizione ottimale da parte di un ampio numero di utenti.

Technology Stack

L'applicazione è stata realizzata utilizzando un moderno stack tecnologico orientato alla scalabilità, alla modularità e alla facilità di sviluppo. Per il frontend è stato scelto **React 19**, che consente la costruzione di un'interfaccia reattiva e componibile, integrato con **React Router 6** per la gestione delle rotte e **React Bootstrap** per un design coerente e responsivo. La parte di styling si basa su **Bootstrap 5** e **Bootstrap Icons**, garantendo accessibilità e coerenza visiva.

Il sistema prevede due backend distinti:

- Un **backend in Java** (SpringBoot), impiegato principalmente per la gestione delle sessioni utente, l'autenticazione e l'integrazione con i servizi di frontend;
- Un **backend in Python**, sviluppato con **FastAPI**, utilizzato per il processamento avanzato dei dati, inclusa l'analisi emozionale tramite modelli di intelligenza artificiale. FastAPI è stato scelto per le sue ottime prestazioni in ambito asincrono e per l'integrazione nativa con OpenAPI.

La gestione della webcam e della registrazione video avviene tramite **WebRTC**, che abilita una comunicazione in tempo reale compatibile con i principali browser, garantendo efficienza nell'acquisizione dei dati visivi.

La sicurezza delle sessioni utente è assicurata tramite **JWT (JSON Web Tokens)**, conservati in **cookie HTTP-only**, per ridurre la superficie d'attacco in caso di vulnerabilità XSS.

Per il testing sono stati utilizzati **Jest** e **React Testing Library**, strumenti ampiamente diffusi nell'ecosistema React che assicurano affidabilità e manutenibilità del codice attraverso test unitari e funzionali.

Infine, per la distribuzione in ambienti di produzione, l'intera applicazione è **containerizzata con Docker** e gestita tramite **Nginx**, con configurazioni automatizzate per **SSL/TLS**, assicurando portabilità, scalabilità e conformità agli standard di sicurezza.

1. Frontend:

- **Linguaggi:** JavaScript (React 19.1.0), HTML, CSS
- **Librerie/Framework:** React Router 6.26.2, React Bootstrap 2.10.9, Bootstrap 5.3.5, Bootstrap Icons 1.13.1
- **Media:** WebRTC per accesso webcam e registrazione video
- **Server web:** Nginx con gestione automatica SSL/TLS.

2. Backend:

- **Java:** gestione autenticazione, sessioni e routing.
- **FastAPI** (Python): analisi emozionale dei video tramite AI.
- **Autenticazione:** JWT con cookie HTTP-only.
- **Web APIs:** WebRTC, WebSocket API, MediaRecorder API, Canvas API.
- **Containerizzazione:** Docker.

3. Database:

- **Database relazionale:** PostgreSQL, integrato tramite JPA/Hibernate nel backend Java
- **Cloud Storage:** Azure Blob Storage per la gestione sicura dei file multimediali

4. Deployment:

- Docker (multi-stage build), Nginx (server di produzione), OpenSSL (generazione certificati SSL)

Diagramma delle Componenti

L'applicazione è progettata secondo un'architettura a livelli, che separa chiaramente le responsabilità per garantire modularità, scalabilità e facilità di manutenzione. Questa suddivisione si articola in tre livelli principali: Frontend, Backend e Database.

1. Frontend

Il livello frontend si occupa della presentazione delle informazioni e dell'interazione diretta con l'utente. Realizzato con **React 19**, utilizza componenti modulari organizzati in cartelle specifiche (components/, pages/, hooks/) e si affida a **React Router 6** per la gestione della navigazione tra le diverse schermate. Il frontend si basa su un design responsivo e mobile-

first, grazie all'adozione di **Bootstrap 5** e **Bootstrap Icons**, per garantire una user experience uniforme e accessibile su diversi dispositivi. In questa parte, vengono gestite tutte le operazioni di profilazione utente, visualizzazione delle immagini, registrazione delle reazioni emotive tramite integrazione **WebRTC** per l'accesso alla webcam e la gestione dei video in tempo reale. Le pagine principali implementano un flusso guidato che comprende l'accoglienza, la raccolta dati demografici, la visualizzazione delle immagini e la registrazione delle emozioni, fino alla presentazione dei risultati.

2.

Backend

Il backend costituisce il nucleo funzionale dell'applicazione e si articola in **due componenti distinti**, sviluppati rispettivamente in **Java (SpringBoot)** e **Python (FastAPI)**, ciascuno con responsabilità specifiche e complementari.

Il **backend Java**, basato su SpringBoot, gestisce principalmente la logica di autenticazione e autorizzazione tramite **JWT** conservati in **cookie HTTP-only**, la gestione sicura delle sessioni utente, il controllo del flusso delle immagini da presentare agli utenti, nonché il coordinamento delle interazioni tra frontend e servizi applicativi.

Il **backend Python** è sviluppato con **FastAPI**, scelto per le sue prestazioni asincrone elevate e il supporto nativo alla documentazione OpenAPI. Questa componente si occupa dell'elaborazione avanzata dei dati video, inclusa l'integrazione con modelli di **intelligenza artificiale per l'analisi emozionale**, il pre-processing dei frame e l'eventuale gestione di task computazionalmente intensivi. Inoltre, funge da proxy verso servizi esterni di AI, garantendo il rispetto delle policy di sicurezza e privacy, in conformità al **GDPR**.

Entrambi i backend sono progettati per essere **scalabili**, **modulari** e facilmente integrabili nell'ambiente di **deployment containerizzato** basato su **Docker**. La gestione delle connessioni sicure (terminazione **SSL/TLS**) e il bilanciamento delle richieste vengono affidati a un **web server Nginx**, che funge da punto di ingresso unificato per l'intero sistema applicativo.

2. Database

Il livello di persistenza dati è responsabile della memorizzazione di tutte le informazioni essenziali al funzionamento dell'applicazione. Conserva i dati relativi agli utenti, come informazioni demografiche e dati di autenticazione, le liste di immagini assegnate per i test emotivi, le registrazioni video (ove previsto), e i risultati dell'analisi emotiva fornita dalle API AI. Il database è progettato per supportare efficienti operazioni CRUD (Create, Read, Update, Delete) necessarie per gestire i dati in modo affidabile e sicuro. Inoltre, garantisce integrità e disponibilità delle informazioni, facilitando l'accesso controllato da parte del backend.

Questa architettura a livelli assicura una chiara separazione delle responsabilità e una gestione modulare dell'applicazione, consentendo aggiornamenti e manutenzioni più semplici, oltre a favorire l'adozione di best practice in termini di sicurezza e performance.

Struttura di Deployment

1. Client Device:

- **Browser Supportati:** L'applicazione è accessibile tramite browser moderni come Google Chrome, Mozilla Firefox, Microsoft Edge e Safari.
- **Pagine Disponibili:** L'utente può accedere alle seguenti sezioni dell'applicazione: Home, pagina di login, Visualizzazione Immagini con Webcam.
- **Protocollo di Comunicazione:** Il traffico avviene tramite HTTPS (porta 443) con fallback su HTTP (porta 80) in ambienti di test o sviluppo.

2. Nginx Web Server (in container Docker):

- **Server HTTP/S:** Nginx serve i file statici della Single Page Application React e funge da reverse proxy verso l'API backend.
- **Gestione SSL:** Supporta HTTPS con certificati SSL/TLS autogenerati tramite OpenSSL.
- **URL Pubblico:** L'applicazione è disponibile all'indirizzo <https://emotions.example.com>.
- **Proxy API Backend:** Le chiamate API vengono inoltrate al server backend tramite configurazione reverse proxy.

3. Frontend Deployment (React App in Docker):

- **Tecnologia:** Applicazione compilata React 19.1.0, servita da Nginx come file statici.
- **Build Output:** Contenuto generato nella directory build/ al momento della compilazione, include JS/CSS/media ottimizzati.
- **Containerizzazione:** Immagine Docker personalizzata multi-stage basata su Node 20-Alpine per build e Nginx-Alpine per produzione.
- **Certificati SSL:** Generati in fase di build tramite OpenSSL usando variabili ambiente configurabili.

4. Backend Server (API in Container Separato o Remoto):

- **Servizio API:** Accessibile tramite reverse proxy da Nginx su percorso /api/.
- **Protocollo di Comunicazione:** HTTPS con gestione token JWT (secure cookie).
- **Integrazione Frontend:** Le chiamate API React sono indirizzate a REACT_APP_API_URL, ad esempio <https://api.emotions.example.com>.

5. Database (Gestito lato Backend):

- **Comunicazione:** Il backend si occupa della persistenza dei dati utente, immagini, risultati di classificazione e sessioni.

6. Sistema di Deployment e Hosting:

- **Docker Compose:** Utilizzato per orchestrare i container frontend (Nginx + React), backend e altri servizi necessari.
- **Server/Cloud Supportato:** Compatibile con deployment su server Linux, VPS (es. DigitalOcean, AWS EC2) o container cloud-native (es. AWS ECS, Azure Container Apps).
- **Porte Aperte:** Porta 80 (HTTP) e 443 (HTTPS) esposte per accesso pubblico.

Flusso delle Operazioni

Il sistema è strutturato in più fasi operative che coinvolgono l'interazione tra frontend, backend, database e un servizio di intelligenza artificiale. Di seguito si descrivono le principali fasi funzionali dell'applicazione.

1. Fase di Registrazione e Login

Al primo accesso alla piattaforma, l'utente interagisce con l'interfaccia frontend, compilando un modulo di registrazione contenente le informazioni personali richieste (es. nome, email, password).

Questi dati vengono inviati al backend, che provvede a verificarne la validità e, nel caso di un nuovo utente, a registrarli nel database. In caso di esito positivo, il database conferma l'avvenuto salvataggio inviando un messaggio di risposta al backend, il quale a sua volta invia un messaggio di conferma al frontend. L'utente viene così autenticato e reindirizzato alla pagina principale.

In caso di login, le credenziali inserite vengono confrontate con quelle già presenti nel database. Se l'autenticazione ha esito positivo, l'utente accede all'applicazione.

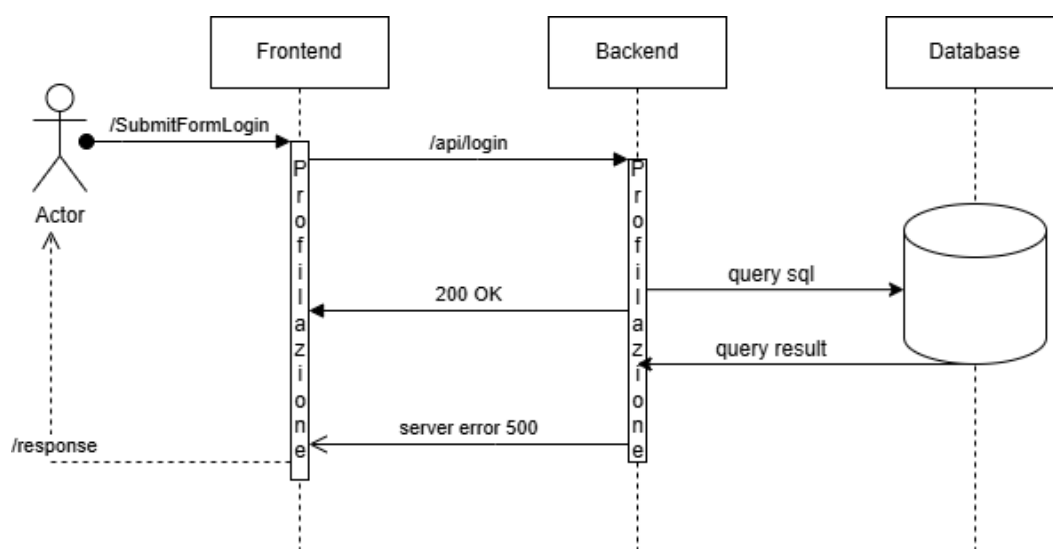


Figura 5: Diagramma di Sequenza per le azioni di Registrazione e Profilazione dei dati utente

2. Fase di Sottomissione delle Risposte

Una volta autenticato, l'utente accede alla schermata principale dell'applicazione. Il frontend effettua una richiesta al backend per ottenere un set di immagini salvate nel database. Una volta ricevute, le immagini vengono mostrate all'utente all'interno dell'interfaccia.

L'utente, per poter procedere, deve autorizzare l'accesso alla webcam. Una volta attivata, viene richiesto all'utente di compilare un modulo in cui inserisce:

- una descrizione soggettiva dell'immagine visualizzata,

- l'emozione percepita in risposta alla visione dell'immagine,
- e viene nel frattempo acquisita un'immagine del volto tramite webcam.

Questi dati vengono raccolti dal frontend e inviati al backend, il quale li inoltra a un modulo di servizio di intelligenza artificiale dedicato all'analisi emotiva. L'AI elabora i dati ricevuti e restituisce al backend una valutazione automatica delle emozioni riconosciute.

Il backend aggrega le informazioni fornite dall'utente e quelle generate dall'AI, quindi procede al salvataggio di tutti i dati nel database. Se il salvataggio va a buon fine, il database invia una risposta di successo al backend, che a sua volta comunica l'esito positivo al frontend.

Il frontend, infine, notifica all'utente l'avvenuta sottomissione dei dati tramite un messaggio di conferma.

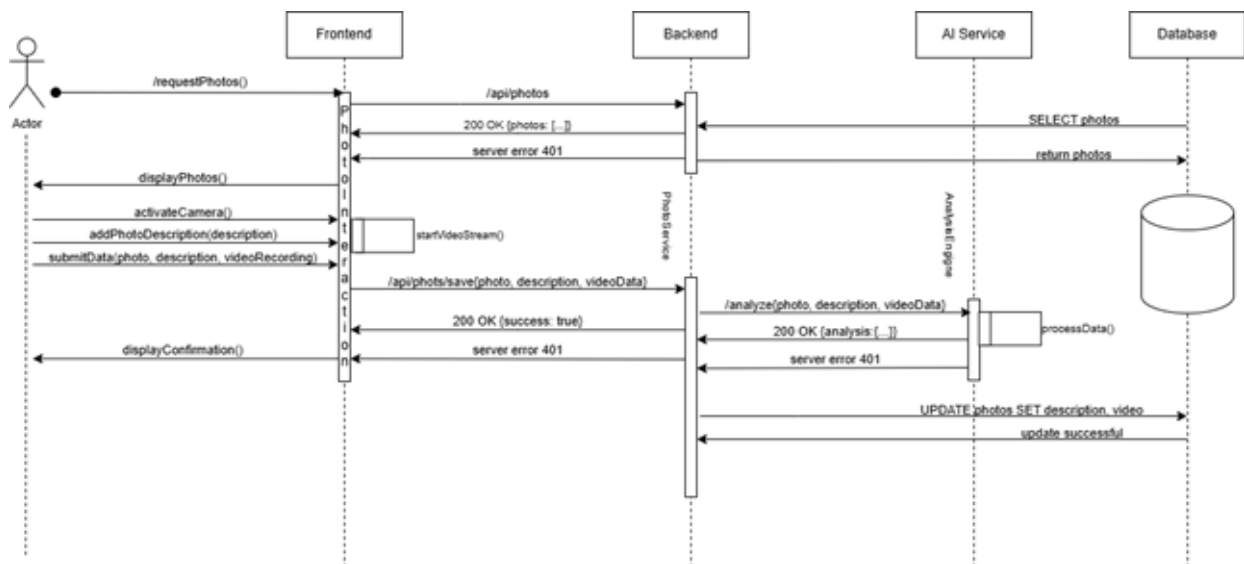


Figura 6: Diagramma delle Squence per le azioni di sottomissione risposte utente

Livello Front-end: Interfaccia Utente

Il livello frontend dell'applicazione costituisce l'interfaccia utente attraverso cui gli utenti interagiscono con il sistema. Realizzato utilizzando il framework **React.js**, il frontend offre una struttura modulare, reattiva e facilmente estendibile. L'interfaccia è progettata per garantire un'esperienza utente intuitiva, accessibile e coerente con i principi di usabilità.

L'utente accede alla piattaforma tramite un browser web, visualizzando una serie di schermate che includono le funzionalità principali del sistema: registrazione, login, visualizzazione delle immagini, attivazione della webcam, inserimento descrizioni e sottomissione delle risposte. Ogni componente dell'interfaccia è stato sviluppato per guidare l'utente in modo semplice e progressivo durante l'interazione, minimizzando l'errore umano e rendendo l'esperienza complessiva fluida.

Il frontend si occupa inoltre di gestire l'acquisizione dell'immagine della webcam in tempo reale e di raccogliere le informazioni inserite dall'utente, inviandole poi in modo sicuro al backend per l'elaborazione. La comunicazione tra frontend e backend avviene tramite richieste HTTP/HTTPS, con gestione asincrona delle risposte, garantendo efficienza e reattività durante l'utilizzo dell'applicazione.

Livello Back-end: Logica Applicativa

Il livello back-end dell'applicazione è suddiviso in due moduli indipendenti, sviluppati in linguaggi differenti per ottimizzare specifici compiti applicativi.

Il primo backend è scritto in **Java (SpringBoot)** e si occupa della gestione dell'autenticazione utente, della logica di sessione, del controllo del flusso degli stimoli visivi (immagini) da presentare e del coordinamento generale tra frontend e servizi esterni. L'autenticazione è realizzata tramite **token JWT (JSON Web Tokens)** memorizzati in **cookie HTTP-only**, al fine di garantire sicurezza contro attacchi XSS e CSRF. Le API RESTful esposte permettono una comunicazione asincrona e modulare con l'interfaccia utente.

Il secondo backend, sviluppato in **Python con il framework FastAPI**, è dedicato all'elaborazione dei contenuti multimediali (in particolare i video registrati dagli utenti) e all'integrazione con servizi di intelligenza artificiale per l'analisi emozionale. FastAPI è stato scelto per la sua capacità di gestire richieste asincrone in modo efficiente, per la leggerezza del framework e per il supporto nativo alla validazione dei dati e alla documentazione automatica delle API. Questo modulo riceve i video, ne cura la pre-elaborazione, li invia al motore AI esterno e restituisce i risultati da salvare a sistema.

Entrambi i backend includono meccanismi di **validazione dei dati**, **gestione centralizzata degli errori**, **logging dettagliato** e **verifica dell'integrità delle richieste**, contribuendo alla robustezza e affidabilità del sistema. L'intera infrastruttura è **containerizzata tramite Docker**, con un **server Nginx** configurato per fungere da reverse proxy e per gestire automaticamente i certificati **SSL/TLS**, garantendo connessioni sicure e conformità alle normative sulla protezione dei dati (es. GDPR).

Struttura Server e Deployment

L'applicazione è progettata per un deployment moderno e scalabile, basato su **contenitori Docker**. L'interfaccia utente, sviluppata con **React**, viene distribuita tramite un contenitore che utilizza **Nginx** come web server di produzione, ottimizzato per servire file statici e gestire richieste HTTPS tramite **certificati SSL generati automaticamente**. Il traffico viene terminato da Nginx, che funge anche da proxy inverso per inoltrare le richieste API verso il back-end Java.

L'intera infrastruttura è definita tramite file **Dockerfile** e **docker-compose.yml**, sfruttando una configurazione multi-stage per ridurre le dimensioni dell'immagine finale e aumentare la sicurezza. Le porte utilizzate sono la **80 (HTTP)** e la **443 (HTTPS)** per l'interfaccia web, mentre il back-end è accessibile tramite la porta **8000**, mappata in locale. Le variabili di ambiente consentono una configurazione flessibile

sia in fase di sviluppo sia in produzione. L'ambiente supporta inoltre l'utilizzo di **WebSocket** per funzionalità in tempo reale, come il tracciamento degli utenti connessi e la classificazione live delle emozioni.

Vantaggi del Tech Stack Selezionato

Il tech stack adottato garantisce una combinazione equilibrata tra modernità, efficienza e scalabilità. Il **front-end in React** consente la costruzione di un'interfaccia utente reattiva e modulare, favorendo lo sviluppo di componenti riutilizzabili e l'adozione di best practices come l'uso del **Context API** per la gestione dello stato globale e di **custom hook** per l'astrazione della logica applicativa. La struttura a componenti rende l'interfaccia facilmente manutenibile, testabile ed estendibile nel tempo.

Il back-end è distribuito su due componenti distinti:

- Un primo modulo, sviluppato in **Java**, gestisce l'autenticazione utente (tramite **JWT con cookie HTTP-only**), la logica di sessione, la distribuzione delle immagini e l'interazione con il front-end tramite API RESTful.
- Un secondo modulo, realizzato in **Python con FastAPI**, è dedicato alla gestione dei contenuti multimediali e all'integrazione con sistemi di **intelligenza artificiale** per l'analisi emozionale. FastAPI consente un'elaborazione asincrona efficiente e una validazione automatica dei dati, con documentazione nativa delle API.

Entrambi i backend adottano meccanismi di **logging strutturato**, **gestione centralizzata degli errori** e **validazione robusta dei dati**, contribuendo a un'infrastruttura solida e resiliente.

Il **deployment containerizzato con Docker** e il reverse proxy **Nginx** assicurano un ambiente coerente e facilmente replicabile tra sviluppo e produzione. La configurazione automatica dei certificati **SSL/TLS** garantisce la sicurezza delle comunicazioni, mentre l'integrazione nativa di **WebSocket** abilita la gestione in tempo reale di eventi e dati, migliorando la reattività dell'interfaccia e la fluidità dell'esperienza utente. Il risultato è un sistema moderno, scalabile e sicuro, in grado di supportare flussi applicativi complessi con un'elevata affidabilità.

Testing

La qualità e l'affidabilità di un sistema complesso come *Emotion Recognition* richiedono un'architettura di testing solida e multilivello, capace di validare sia le unità funzionali che l'interazione tra i vari componenti software.

La strategia adottata per il progetto prevede **unit test**, **integration test** e **test end-to-end**, utilizzando tecnologie specifiche per ciascun modulo dell'applicazione.

Testing del Backend: JUnit e TestContainers

Il backend, sviluppato in Java con Spring Boot, è testato principalmente attraverso **unit test scritti con JUnit 5**. Ogni componente viene verificato in isolamento grazie all'uso di **mocking con Mockito**, così da assicurare la correttezza della logica applicativa senza dipendenze esterne.

Un esempio emblematico è la classe di test `AppControllerTest`, nella quale vengono verificati i principali endpoint REST del controller. In questo contesto, i servizi `UserService`, `AzureStorageService` e `ReactionsService` vengono simulati tramite oggetti mock, permettendo così di testare il comportamento del controller in risposta a specifici scenari. I metodi testati includono operazioni critiche come login, logout, `registerResult` e `downloadImage`, ognuno dei quali restituisce risposte HTTP validate tramite asserzioni sui codici di stato e sui contenuti delle risposte.

Per quanto riguarda l'integrazione con il database, si fa uso di **TestContainers**, una tecnologia che consente l'esecuzione di test di integrazione in ambienti containerizzati e realistici, con istanze effimere di PostgreSQL. Questo approccio permette di replicare fedelmente le condizioni del runtime produttivo, testando persistenza, vincoli e transazioni su dati reali senza compromettere l'ambiente locale o condiviso.

Testing del Modulo Video Classifier: FastAPI TestClient

Il modulo dedicato all'analisi video, basato su Python e FastAPI, adotta una strategia parallela per garantire la qualità del codice. A livello unitario, viene utilizzato **pytest**, uno dei framework di testing più diffusi in ambito Python, per testare le funzioni core del classificatore video, come l'analisi delle espressioni facciali e la decodifica dei frame.

Per verificare l'esposizione dei servizi REST del microservizio video, viene invece adottato il **TestClient di FastAPI**, che consente di simulare chiamate HTTP verso l'API locale e verificarne il comportamento in condizioni controllate. Questo strumento è fondamentale per assicurare la correttezza delle interfacce e la coerenza nei dati scambiati tra i servizi.

End-to-End Testing: Integrazione Completa

A completamento della strategia, sono stati definiti test di tipo **end-to-end** in grado di validare l'interazione tra tutti i componenti del sistema. Questi test orchestrano l'intero flusso applicativo, dalla creazione del profilo utente, alla somministrazione delle immagini, fino alla raccolta e analisi delle reazioni, verificando che la comunicazione tra backend Java e microservizio Python avvenga correttamente.

Questi test integrati vengono eseguiti in ambienti simulati o containerizzati, in cui i due servizi sono avviati simultaneamente. Si tratta di una fase cruciale per rilevare anomalie legate a incompatibilità di interfaccia, gestione dei token, latenza nei servizi esterni o errori nel flusso dati multimodale.

In conclusione dunque, la strategia di testing adottata per Emotion Recognition riflette la complessità e l'interdisciplinarietà del progetto. Attraverso l'uso combinato di test unitari, di integrazione e end-to-end, e mediante l'impiego di strumenti avanzati come JUnit, TestContainers, pytest e TestClient, il team ha potuto assicurare una copertura funzionale robusta, ridurre i regressi e garantire l'affidabilità del sistema nelle condizioni operative previste. L'approccio modulare, flessibile e riproducibile dei test consente inoltre di mantenere elevati standard qualitativi anche in fase di evoluzione e aggiornamento della piattaforma.

Validazione dell' Applicativo Run-Time

Home Page

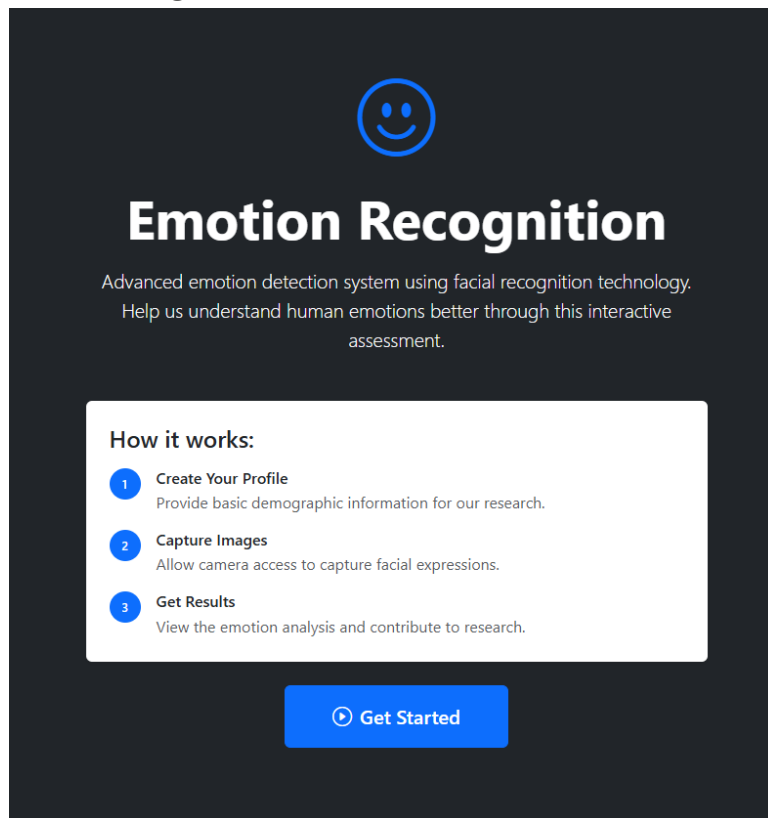


Figura 7: Screen della HomePage dell'applicativo sviluppato

La Home Page rappresenta l'accesso iniziale alla piattaforma *Emotion Recognition* e ha lo scopo di presentare il progetto in modo chiaro, accessibile e guidato, migliorando la user experience fin dai primi secondi di interazione. L'intero contenuto della pagina è incapsulato nel componente `HomePage.jsx`, con uno stile coerente al tema dark della piattaforma, garantito dallo stylesheet associato.

La pagina è suddivisa in diverse sezioni principali, tutte contenute in un layout responsive fornito da react-bootstrap:

- [Intestazione introduttiva](#)

Inserita all'interno di un `<div className="text-center text-white">`, introduce il nome dell'applicazione con un'icona visiva (bi-emoji-smile), un titolo principale e una breve descrizione della funzionalità: il sistema di riconoscimento delle emozioni tramite espressioni facciali. Viene anche fornito un contesto che chiarisce il valore del contributo dell'utente nella ricerca.

```

22     <div className="text-center text-white mb-5">
23       <div className="mb-4">
24         <i className="bi bi-emoji-smile display-1 text-primary"></i>
25       </div>
26       <h1 className="display-4 fw-bold mb-3">Emotion Recognition</h1>
27       <p className="lead mb-4">
28         Advanced emotion detection system using facial recognition technology.
29         Help us understand human emotions better through this interactive assessment.
30       </p>
31     </div>

```

Figura 8: Esempio di linee di codice React per costruzione del blocco di titolo e breve abstract del progetto

- Sezione “How it works”

Inserita in un componente Card, questa sezione guida l'utente nei tre step fondamentali:

1. Creazione del profilo – Raccolta di informazioni demografiche di base.
2. Acquisizione immagini – Attivazione della webcam per rilevare le espressioni facciali.
3. Visualizzazione dei risultati – Analisi delle emozioni e restituzione dell'output.

Il layout è realizzato con d-flex e badge numerati per enfatizzare la sequenzialità delle azioni.

```

33     <Card className="mb-4">
34       <Card.Body className="p-4">
35         <h3 className="h4 mb-3">How it works:</h3>
36         <div className="d-flex flex-column gap-3">
37           <div className="d-flex align-items-start">
38             <div className="badge bg-primary rounded-circle me-3 d-flex align-items-center justify-content-center" style={{width: '2rem', height: '2rem'}}>
39               1
40             </div>
41             <div>
42               <h6 className="mb-1">Create Your Profile</h6>
43               <p className="mb-0 text-muted">Provide basic demographic information for our research.</p>
44             </div>
45           </div>
46           <div className="d-flex align-items-start">
47             <div className="badge bg-primary rounded-circle me-3 d-flex align-items-center justify-content-center" style={{width: '2rem', height: '2rem'}}>
48               2
49             </div>
50             <div>
51               <h6 className="mb-1">Capture Images</h6>
52               <p className="mb-0 text-muted">Allow camera access to capture facial expressions.</p>
53             </div>
54           </div>
55           <div className="d-flex align-items-start">
56             <div className="badge bg-primary rounded-circle me-3 d-flex align-items-center justify-content-center" style={{width: '2rem', height: '2rem'}}>
57               3
58             </div>
59             <div>
60               <h6 className="mb-1">Get Results</h6>
61               <p className="mb-0 text-muted">View the emotion analysis and contribute to research.</p>
62             </div>
63           </div>
64         </div>
65       </Card.Body>
66     </Card>

```

Figura 9: Esempio di linee di codice React per costruzione del box principale visibile nella HomePage

- [Avviso di compatibilità browser](#)

Se il browser non supporta le funzionalità richieste (es. accesso alla fotocamera), viene mostrato un Card con icona di avvertimento (bi-exclamation-triangle-fill), grazie alla funzione `checkBrowserSupport()`. Questo aumenta la robustezza dell'interfaccia prevenendo malfunzionamenti in ambienti non compatibili.

```

68      {!browserSupport.isSupported && (
69        <Card className="border-warning mb-4">
70          <Card.Body>
71            <div className="d-flex align-items-center">
72              <i className="bi bi-exclamation-triangle-fill text-warning me-3 fs-4"></i>
73              <div>
74                <h6 className="mb-1">Browser Compatibility</h6>
75                <p className="mb-0 text-muted">
76                  Your browser may not support all required features. For the best experience,
77                  please use a modern browser with camera access enabled.
78                </p>
79              </div>
80            </div>
81          </Card.Body>
82        </Card>

```

Figura 10: Esempio di linee di codice React utili per avvisare l'utente in caso di mancata compatibilità del browser con le funzionalità richieste dall'applicativo

- [Call to Action e Informativa sulla Privacy](#)

Il pulsante "Get Started", posizionato centralmente, richiama la funzione `handleGetStarted` che reimposta lo stato globale e reindirizza l'utente al profilo (`ROUTES.PROFILE`). Viene anche visualizzato un messaggio sul trattamento dei dati ("Your privacy is important to us..."), sottolineando la conformità alle buone pratiche etiche e di sicurezza.

```

82      )} <div className="text-center">
83        <Button
84          variant="primary" |
85          size="lg"
86          onClick={handleGetStarted}
87          className="px-5 py-3"
88        >
89          <i className="bi bi-play-circle me-2"></i>
90          Get Started
91        </Button>
92      </div>

```

Figura 11: Esempio di linee di codice React per costruzione Button GetStarted

Aspetti tecnici e di design

- [Design coerente e responsivo](#)

L'intera pagina sfrutta componenti react-bootstrap, assicurando una resa omogenea su dispositivi mobili e desktop, e utilizza classi Bootstrap Icons per garantire una UI coerente.

- [Accessibilità e compatibilità](#)

Il tema dark ad alto contrasto, le icone testuali, e la disposizione lineare dei contenuti rendono la pagina fruibile anche a utenti con esigenze speciali. L'infrastruttura supporta la navigazione da tastiera e la compatibilità con screen reader.

- [Gestione stato globale](#)

Il componente accede al contesto globale tramite `useAppContext()` per resettare lo stato dell'app all'avvio, assicurando un'esperienza pulita e priva di conflitti con dati residui.

Profile Page

La procedura di autenticazione dell'utente all'interno dell'applicazione è implementata attraverso la pagina `ProfilePage`, che consente la raccolta e la validazione dei dati anagrafici dell'utente. Il componente, realizzato con React, utilizza gli hook `useState` per la gestione dello stato locale del profilo (`localProfile`) e `useNavigate` per la navigazione programmata.

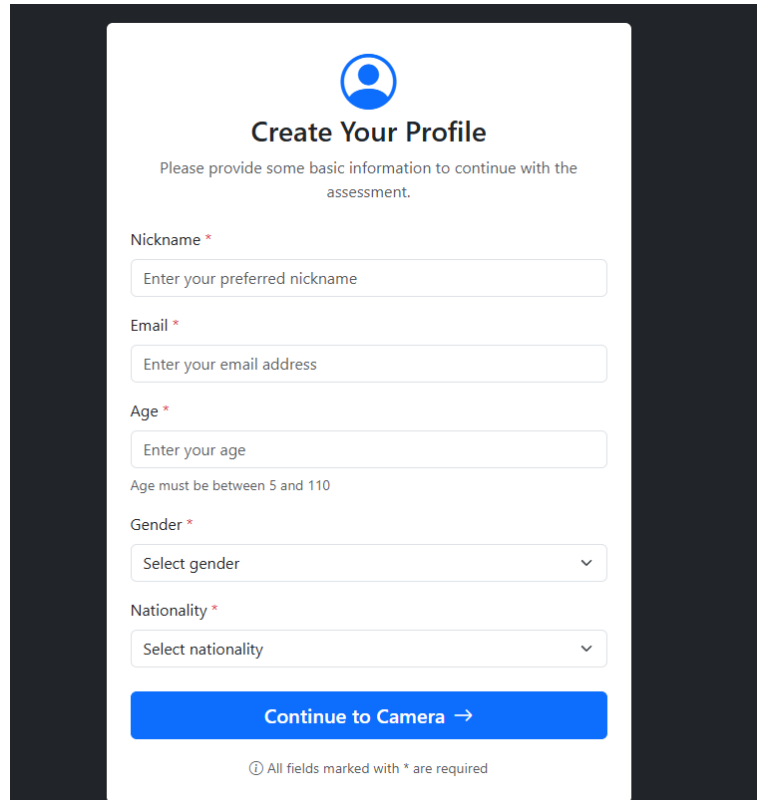


Figura 12: Screen della `ProfilePage` dell'applicativo sviluppato

```
11  const ProfilePage = () => {
12    const { state, setProfile, setImages, setAuthenticated } = useAppContext();
13    const [localProfile, setLocalProfile] = useState(state.profile);
14    const [errors, setErrors] = useState({}); const [isSubmitting, setIsSubmitting] = useState(false);
15    const navigate = useNavigate();
```

Figura 13: Esempio di linee di codice React per inizializzazione stati

L'autenticazione è concepita in forma semplificata: non si basa su credenziali tradizionali (username/password), bensì sulla validazione di un insieme di dati personali obbligatori (*nickname*, *email*, *età*, *genere*, *nazionalità*). L'inserimento è gestito tramite il form di React Bootstrap, i cui cambiamenti vengono intercettati dalla funzione `handleChange`, che aggiorna lo stato locale e rimuove eventuali errori dal campo modificato:

```
17  ✓    const handleChange = (e) => {  
18      const { name, value } = e.target;  
19  
20      // Convert age to integer for number input  
21      const processedValue = name === 'age' ? (value === '' ? null : parseInt(value, 10)) : value;  
22  
23      setLocalProfile({ ...localProfile, [name]: processedValue });  
24  
25      // Clear error for this field when user starts typing  
26      if (errors[name]) {  
27        setErrors({ ...errors, [name]: '' });  
28      }  
29    };
```

Figura 14: Esempio di linee di codice React in cui avviene controllo sul campo "age" compilato dall'utente

Il flusso prevede l'inserimento dei dati in un form strutturato tramite componenti `Form`, `Form.Group`, `Form.Control`, come nel seguente esempio per il campo `nickname`:

```
94      <Form onSubmit={handleSubmit}>  
95        <Form.Group className="mb-3">  
96          <Form.Label>Nickname <span className="text-danger">*</span></Form.Label>  
97          <Form.Control  
98            type="text"  
99            name="nickname"  
100            value={localProfile.nickname}  
101            onChange={handleChange}  
102            isValid={!errors.nickname}  
103            placeholder="Enter your preferred nickname"  
104          />  
105          <Form.Control.Feedback type="invalid">  
106            {errors.nickname}  
107          </Form.Control.Feedback>  
108        </Form.Group>
```

Figura 15: Esempio di linee di codice React per costruzione form

La validazione lato client è realizzata mediante la funzione `validateProfile`. In caso di esito positivo, i dati vengono inviati al server con `apiService.submitProfile` nella funzione `handleSubmit`:


```
31  const handleSubmit = async (e) => {  
32    e.preventDefault();  
33    setIsSubmitting(true);    const validation = validateProfile(localProfile);  
34    |  
35    if (!validation.isValid) {  
36      setErrors(validation.errors);  
37      setIsSubmitting(false);  
38      return;  
39    }
```

Figura 16: Esempio di linee di codice React con funzione asincrona per validazione profile

Durante l'elaborazione della richiesta, viene mostrato un indicatore di caricamento per migliorare l'esperienza utente:

```
70    if (isSubmitting) {  
71      return <LoadingSpinner message="Saving your profile..." fullScreen />;  
72    }
```

Figura 17: Esempio di linee di codice React per costruzione LoadingSpinner

Una volta completata l'autenticazione e salvati i dati nel contesto globale (setProfile, setImages, setAuthenticated(true)), l'utente viene automaticamente reindirizzato alla schermata successiva (ROUTES.IMAGE), dedicata alla fase di acquisizione delle immagini tramite webcam.

Il form integra inoltre meccanismi di feedback visuale per eventuali errori di input (isInvalid, Form.Control.Feedback) e messaggi di errore globali in caso di fallimento della richiesta.

È garantita anche la compatibilità con i vincoli di età definiti tramite la costante AGE_CONSTRAINTS e la selezione dei valori da liste predefinite come GENDER_OPTIONS e countryList.

Image Page

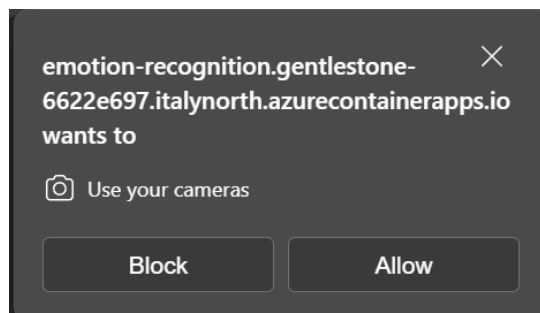


Figura 18: Esempio di alert per richiesta accesso a videocamera del dispositivo

La pagina **Camera Capture** è il cuore dell'esperienza utente, dedicata alla cattura e all'annotazione emotiva di una serie di immagini mostrate all'utente, combinando l'analisi delle reazioni video, input manuali ed elaborazione tramite AI.

L'utente viene guidato in un processo iterativo: per ciascuna immagine, viene attivata automaticamente la webcam, viene avviata una breve registrazione, e infine l'utente seleziona una o più emozioni e può inserire un commento descrittivo.

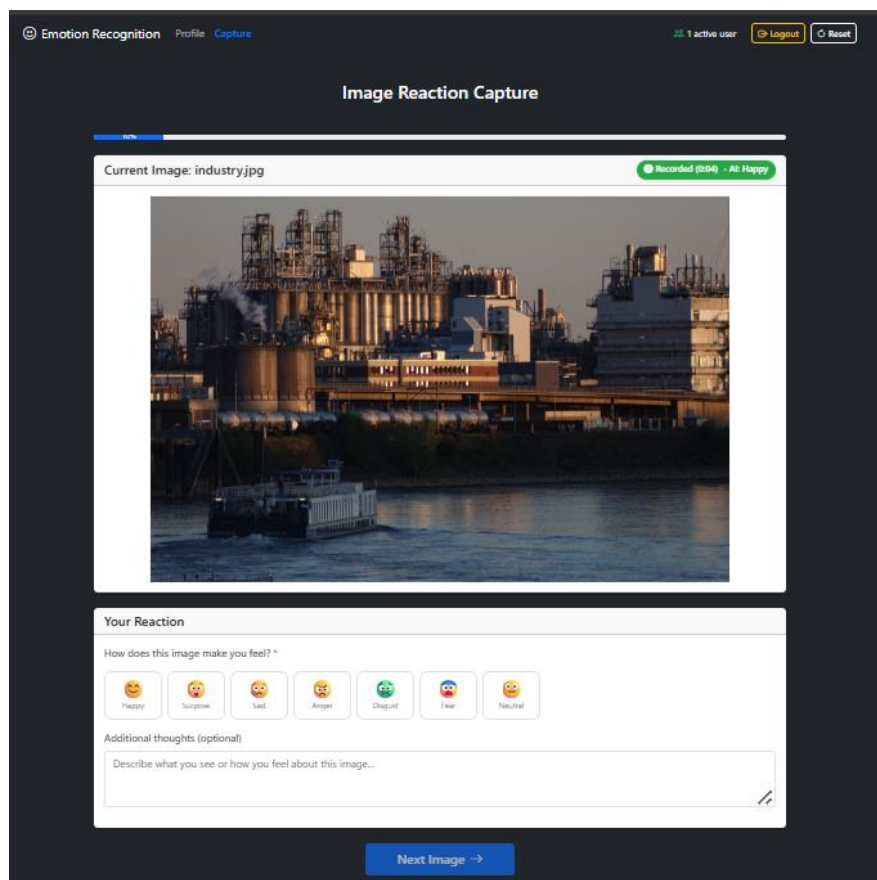


Figura 19: Screen della ImagePage dell'applicativo sviluppato

La webcam viene avviata all'ingresso nella pagina usando un hook personalizzato:

```
125      // Start webcam
126      useEffect(() => {
127          startWebcam();
128          return () => {
129              stopWebcam();
130          };
131      }, [startWebcam, stopWebcam]);
```

Figura 20: Esempio di linee di codice React con hook per avvio e disattivazione della videocamera

Il flusso di registrazione prevede inizializzazione, cattura dei chunk e gestione dello stop automatico:

```
247      mediaRecorder.start();
248      setIsRecording(true);
249      setRecordingDuration(0);
250      setError(null);
251      |
252      // Start timer
253      recordingTimerRef.current = setInterval(() => {
254          setRecordingDuration(prev => prev + 1);
255      }, 1000);
256
257      // Auto-stop recording after 4 seconds
258      autoStopTimerRef.current = setTimeout(() => {
259          if (mediaRecorderRef.current && mediaRecorderRef.current.state === 'recording') {
260              stopRecording();
261          }
262      }, 4000);
```

Figura 21: Esempio di linee di codice React per avvio della registrazione video

Al termine della registrazione, il video viene automaticamente inviato all'AI per una classificazione delle emozioni. La previsione AI viene poi mostrata all'utente in un badge accanto alla registrazione.

```
336 // Process video for AI emotion prediction
337 const processVideoForPrediction = async (videoBlob) => {
338   try {
339     setIsProcessingVideo(true);
340     setError(null);
341
342     // Call the API to get emotion prediction
343     const emotionResponse = await ApiService.submitRecognition(videoBlob);
344     const emotion = emotionResponse.emotion || 'unknown';
```

Figura 22: Esempio di linee di codice React con chiamata all'API responsabile del riconoscimento di emozioni nel video registrato del volto dell'utente

In questa schermata inoltre, l'utente può selezionare, in base all'emozione che la foto mostrata dal sito scatuisce in lui, una o più emozioni da una lista predefinita, ciascuna rappresentata da un'emoji. Quando l'utente clicca su un'emoji, questa viene evidenziata, indicando che è stata selezionata ed analogamente se l'utente clicca di nuovo sulla stessa emoji, questa si deseleziona.

In seguito, le emozioni selezionate vengono salvate nello stato interno del componente, quindi sono immediatamente visibili all'utente e pronte per essere inviate al backend.

```
49 // Available emotion options
50 const emotionOptions = [
51   { value: 'happy', emoji: '😊', label: 'Happy' },
52   { value: 'surprise', emoji: '😮', label: 'Surprise' },
53   { value: 'sad', emoji: '😞', label: 'Sad' },
54   { value: 'anger', emoji: '😡', label: 'Anger' },
55   { value: 'disgust', emoji: '😬', label: 'Disgust' },
56   { value: 'fear', emoji: '😱', label: 'Fear' },
57   { value: 'neutral', emoji: '😐', label: 'Neutral' }
58 ];
59
60 // Handle emotion selection
61 const handleEmotionToggle = (emotionValue) => {
62   setSelectedEmotions(prev => {
63     if (prev.includes(emotionValue)) {
64       return prev.filter(emotion => emotion !== emotionValue);
65     } else {
66       return [...prev, emotionValue];
67     }
68   });
69 };
```

Figura 23: Esempio di linee di codice React per costruzione array con le varie emozioni selezionabili dall'utente

Inoltre, è possibile aggiungere un commento testuale che viene analizzato in tempo reale via WebSocket:

```
71 // Handle comment changes and send real-time emotion classification with debouncing
72 ✓ const handleCommentChange = (e) => {
73   const newComment = e.target.value;
74   setComment(newComment);
```

Figura 24: Esempio di linee di codice React per gestione commento

```
3   if (newComment.trim().length > 0) {
4     debounceTimeoutRef.current = setTimeout(() => {
5       websocket?.sendText(newComment.trim());
6     }, 300); // 300ms debounce delay for optimal real-time experience
7   }
8   };
```

Figura 25: Esempio di linee di codice React per pulizia e controllo commento inserito dall'utente, in particolare si effettuano controlli per accertarsi che questo non abbia lunghezza nulla

Infine, tutti i dati (video, emozioni, commento, AI prediction e classificazione real-time) vengono consolidati e salvati come reazione all'immagine corrente e l'app mostrerà automaticamente la prossima immagine all'utente.

```
385 const reactionData = {
386   imageId: state.images[state.currentIndex],
387   description: comment.trim() || '',
388   reaction: emotion !== 'unknown' ? emotion : (selectedEmotions.length > 0 ? selectedEmotions[0] : 'neutral'),
389   aiComment: emotion,
390   selectedEmotions: selectedEmotions, // Store user-selected emotions
391   sentimentAnalysis: websocket?.emotionClassification, // Store real-time sentiment analysis
392   videoBlob: recordedVideo,
393   timestamp: new Date().toISOString()
394 };
```

Figura 26: Esempio di linee di codice React per costruzione oggetto con raccolta dei dati sottomessi dall'utente per ogni immagine visualizzata nella ImagePage

La pagina CameraCapture include meccanismi espliciti di pulizia (cleanup) per evitare che, al cambio di immagine o durante lo smontaggio del componente (es. quando si naviga via), vengano lasciate risorse non più necessarie attive. Questo è importante per:

- Evitare memory leak (perdita di memoria), per cui il componente non lascia attivi timer, video, o URL inutilizzati.
- Liberare risorse come lo stream della webcam o oggetti URL temporanei.
- Evitare timer inutili che continuano a eseguire codice dopo che il componente è stato dismesso.

FINAL PAGE

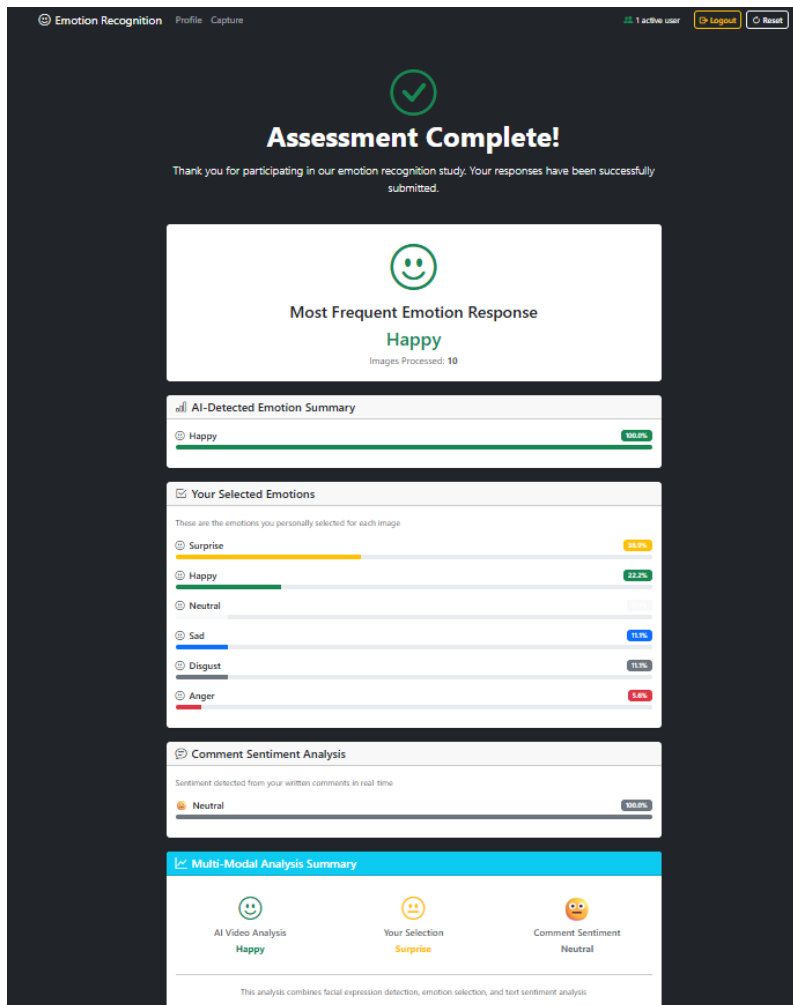


Figura 27: Screen della FinalPage dell'applicativo sviluppato

La **FinalPage** rappresenta la schermata conclusiva del processo di valutazione delle emozioni, dove vengono raccolti, elaborati e visualizzati i risultati ottenuti. Al momento del montaggio del componente (useEffect), viene automaticamente inviata la valutazione finale tramite l'endpoint submitAssessment di ApiService. Prima dell'invio, viene verificato che l'utente sia autenticato e che ci siano reazioni valide (if (!userId || !hasReactions || !isAuthenticated)), altrimenti si viene reindirizzati alla homepage (navigate(ROUTES.HOME)).

```

41      // Prepare the assessment data
42      const assessmentData = {
43        imagesDescriptionsAndReactions: imageReactions.map(reaction => ({
44          image: reaction.imageId,
45          description: reaction.description || '',
46          reaction: reaction.reaction,
47          aiComment: reaction.aiComment || reaction.reaction
48        })))
49      };

```

Figura 28: Esempio di linee di codice React per costruzione oggetto contenente i dati raccolti necessari per effettuare analisi valutative

Dopo l'invio, viene costruito un riepilogo statistico che comprende l'emozione predominante rilevata dall'AI (dominantEmotion), quella selezionata più frequentemente dall'utente (dominantSelectedEmotion), e il sentimento prevalente nei commenti testuali (dominantSentiment). Vengono inoltre calcolate le distribuzioni percentuali per ciascuna categoria tramite riduzioni sui conteggi (emotionCounts, selectedEmotionCounts, sentimentCounts).

```
79         const dominantEmotion = Object.keys(emotionCounts).reduce((a, b) =>
80             emotionCounts[a] > emotionCounts[b] ? a : b
81         );
```

Figura 29: Esempio di linee di codice React per calcolo emozione dominante

La UI mostra in modo interattivo queste statistiche, attraverso progress bar, badge colorati (derivati da getEmotionColor e getSentimentColor), e icone (getEmotionIcon, getSentimentIcon) per ogni emozione o sentimento rilevato. Ogni reazione individuale è visualizzata con dettagli come: l'emozione AI, quelle selezionate dall'utente, il commento scritto e il relativo sentiment analizzato.

Un'ulteriore sezione Multi-Modal Analysis Summary confronta direttamente le tre dimensioni di analisi (AI, utente, e testo) fornendo una visione integrata del comportamento emotivo.

Infine, viene fornita una sezione con informazioni di sessione (es. nickname, user ID, timestamp) e un bottone per ricominciare la valutazione (handleStartOver), che resetta lo stato e riporta alla homepage.

Il WebSocket viene disconnesso in modo pulito al momento della disattivazione del componente (useEffect con return () => {...}), garantendo che non vi siano connessioni aperte non più necessarie.

Conclusioni e sviluppi futuri

Il progetto sviluppato ha dimostrato la fattibilità di una piattaforma di analisi multimodale delle emozioni, basata su immagini, descrizioni testuali e reazioni utente. L'integrazione di tecniche di intelligenza artificiale per la classificazione emotiva (tramite modelli preaddestrati) con la valutazione soggettiva da parte dell'utente ha permesso di generare un quadro ricco e stratificato della risposta emozionale a stimoli visivi.

Dal punto di vista tecnico, l'applicazione si distingue per l'impiego combinato di diversi paradigmi:

- raccolta dati in tempo reale tramite WebSocket,
- visualizzazione dinamica attraverso componenti React con stato globale gestito da context,
- elaborazione asincrona dei dati con API REST,
- calcolo statistico distribuito lato client.

Il sistema, inoltre, è stato progettato per mantenere un buon grado di scalabilità e modularità, rendendolo estendibile a contesti di utilizzo diversi, come la valutazione dell'efficacia comunicativa in campagne pubblicitarie, l'analisi psicologica individuale o lo studio delle risposte affettive a contenuti generati artificialmente.

Tuttavia, l'implementazione attuale presenta alcune limitazioni. Anzitutto, la classificazione delle emozioni da parte dell'AI è affidata a una pipeline statica e generalizzata, non ottimizzata per domini specifici o per la popolazione target. Inoltre, l'analisi dei sentimenti sui commenti testuali è soggetta a imprecisioni dovute alla ridotta granularità semantica dei modelli utilizzati, che potrebbero non cogliere sfumature ironiche, ambivalenti o culturalmente connotate. Anche la mancanza di una validazione scientifica dei risultati raccolti limita, al momento, l'affidabilità dell'applicazione in contesti clinici o accademici rigorosi.

Per superare tali criticità, si prospettano alcune linee di sviluppo:

1. **Personalizzazione dei modelli emotivi:** Sostituire o affiancare i modelli generici con modelli adattivi, addestrati su dataset raccolti all'interno dell'app stessa, potrebbe migliorare la pertinenza delle analisi. Inoltre, l'introduzione di modelli di deep learning fine-tuned per domini specifici (es. educazione, marketing, salute mentale) rappresenta una direzione promettente.
2. **Integrazione di segnali biometrici:** L'aggiunta di dati provenienti da fonti fisiologiche (come espressioni facciali in tempo reale, tassi cardiaci, GSR) permetterebbe un'analisi più oggettiva e multidimensionale della risposta emotiva.
3. **Validazione sperimentale:** È auspicabile una fase di testing controllato con gruppi di utenti reali e metriche psicometriche standard, per valutare l'affidabilità e la coerenza tra l'output del sistema e le risposte soggettive dichiarate.
4. **Ottimizzazione dell'esperienza utente (UX):** Il flusso dell'applicazione può essere ulteriormente raffinato attraverso test di usabilità, in particolare per migliorare l'engagement nelle fasi di inserimento dei commenti e nelle visualizzazioni statistiche finali.

5. **Esportabilità dei risultati:** L'implementazione di un sistema di esportazione (in PDF o CSV) dei risultati della sessione consentirebbe una maggiore fruibilità dell'analisi anche al di fuori dell'interfaccia web.

In sintesi, questo lavoro rappresenta un primo passo verso una piattaforma interattiva per l'analisi emotiva multimodale, capace di coniugare automazione, interpretabilità e coinvolgimento dell'utente. I margini di miglioramento sono ampi, ma le basi tecniche e concettuali poste risultano solide e promettenti.

Bibliografica

- **Modello di Computer Vision:** [ResEmoteNet: Bridging Accuracy and Loss Reduction in Facial Emotion Recognition | Papers With Code](#)
- **Grand View Research** per dati su affective computing (2023–2030) [imarcgroup.com+4grandviewresearch.com+4industryarc.com+4](#)
- **Grand View Research** su emotion recognition (2023–2030) [researchandmarkets.com](#)
- **Roots Analysis** su emotion detection & recognition (2025–2035) [businessresearchinsights.com+7rootsanalysis.com+7linkedin.com+7](#)
- **SNS Insider, IMARC Group, Verified Market Research** su affective computing e emotion recognition (2023–2032/33) [imarcgroup.com+2verifiedmarketresearch.com+2snsinsider.com+2](#)
- **MarketsandMarkets, GMI Insights, MarketandMarkets, Verified Market Research** su emotion AI (2024–2030/2032/2034) [verifiedmarketresearch.com+2gminsights.com+2marketsandmarkets.com+2](#)